LoRA 파라미터 구조 이해

비유로 배우는 지도 : 무엇을, 어디에, 얼마나?

Catastrophic Forgetting to SFT, PEFT, LoRA

초보자를 위한 질문과 답으로 배우는 LoRA 가이드

들어가는 말

LLM 전체를 다시 학습시키는 Full Fine-Tuning 은 새로운 지식을 얻는 대신 기존에 쌓아둔 일반 지식을 잃어버리는 Catastrophic Forgetting 위험이 크기 때문에, 원본 파라미터는 그대로 동결하고 극히 일부만 학습시키는 PEFT (Parameter Efficient Fine Tuning) 방식으로 무게중심이 옮겨가고 있습니다.

책 전체를 다시 써버리는 Full Fine-Tuning 은 새 내용을 담는 대신 원문이 가진 지식을 지워버리는 Catastrophic Forgetting 위험이 있기 때문에, 원문은 그대로 두고 포스트잇만 새로 붙이는 PEFT(LoRA) 방식이 실전에서 선호됩니다.

SFT (Supervised Fine Tuning : 지도학습 파인튜닝)
= "정답이 있는 데이터로 모델을 미세조정한다"는 목적/방법론 그 자체

↓ 이걸 어떻게 실행할 것인가?

┌── Full Fine-Tuning → 전체 파라미터를 다시 학습 (Catastrophic Forgetting 위험)
└── **PEFT** → 원본은 동결, 극히 일부만 추가로 학습 (효율적, 실전 선호)

↓ PEFT 안에도 여러 세부 기법이 있음

Prefix-Tuning · Prompt-Tuning · Adapter Tuning · LoRA · QLoRA · DoRA ...

↓

LoRA ← 이 가이드가 다루는 기법

sLLM Fine-Tuning 에는 Full Fine-Tuning, Prefix-Tuning, Adapter Tuning, QLoRA 등 다양한 이론과 알고리즘이 존재합니다. 이 가이드가 그중 LoRA 에 집중하는 이유는, 우리의 목적이 이론 자체를 연구하는 것이 아니라 실전에 적용하는 것이기 때문입니다. LoRA 는 그중에서도 현업에서 가장 널리 쓰이는 기법이며, 이 구조를 정확히 이해하면 다른 파생 기법까지 자연스럽게 확장해서 이해할 수 있습니다.

이유는 두 가지입니다.

첫째, sLLM Fine-Tuning 이론 지형은 Full Fine-Tuning, Prefix-Tuning, Prompt-Tuning, Adapter Tuning, LoRA, QLoRA, DoRA 등으로 매우 넓습니다. 이걸 전부 학습하는 것은 연구자의 목적이지, 실무에서 모델을 현장에 적용하려는 목적과는 다릅니다.

둘째, 그중에서도 LoRA 는 실제 현업에서 가장 널리 쓰이는 기법입니다. 적은 GPU 메모리로 학습이 가능하고, HuggingFace PEFT 같은 표준 도구 생태계가 갖춰져 있으며, QLoRA·DoRA 등 대부분의 실전 파생 기법이 LoRA 구조를 그대로 확장한 것이기 때문입니다. 즉 LoRA 하나를 정확히 이해하면 파생 기법으로 확장하기가 쉽습니다.

전체 지도 한눈에 보기

이 가이드는 Qwen2.5-1.5B 로 LoRA 실습을 할 때 자연스럽게 떠오르는 질문들을 순서대로 따라갑니다. 아래 지도를 먼저 보고, 각 질문이 왜 이 순서로 배치되었는지 확인한 뒤 본문으로 들어가겠습니다.

자, 이제 sLLM 실무와 LoRA 이해를 위한 여행을 떠나 볼까요?

Q1·Q2 · 무엇을 새로 추가하는가? (개념)

↓

Q3 · 왜 그래도 되는가? (저랭크 가설)

↓

Q4·Q5 · 그 값은 언제, 어떻게 정해지는가? (구조 vs 학습)

↓

Q6·Q7 · 어디에 붙이는가? (위치)

↓

Q8·Q9 · Q/K/V 와 적용 범위

↓

Q10 · 왜 층마다 따로 학습되는가? (구조)

↓

Q11·Q12 · 얼마나 크게 만들 것인가? (Rank)

↓

Q13 · 한 문장 요약

↓

Q14 · 여러 개를 동시에 쓸 수 있는가? (다중 Adapter)

↓

Q15 · 파라미터는 항상 재현되는가? (재현성)

↓

Q16 · 결국 무엇이 성능을 가르는가? (데이터·도메인 지식)

Group	구분	핵심 질문	쓰인 비유	핵심 개념
개념 이해	Q1	일부 파라미터를 고르는 건가?	포스트잇 메모	Frozen vs Trainable
	Q2	병렬 구조인가?	우회 도로	LoRA A→B, 출력 합산
이론적 근거	Q3	왜 그래도 되는가?	완성된 벽화 위 트레이싱지	저랭크 가설(Low Intrinsic Rank)
값의 결정	Q4	새 파라미터 값은 어떻게 정해지나?	트레이싱지에 연필로 스케치	초기화(A=무작위,B=0) + 학습
	Q5	붙이자마자 개수가 왜 줄어 보이나?	빈 트레이싱지를 테이프로 붙임	구조 생성 시점 vs 값 학습 시점
적용 위치	Q6	전체에 다 붙이나?	건물 리모델링	target_modules
	Q7	위치는 누가 정하나?	이미 지어진 건물	Qwen 아키텍처 vs 사용자 선택
Q/K/V · 적용 범위	Q8	Q/K/V 는 뭔가?	도서관 검색	Query / Key / Value
	Q9	왜 q, v 만 골랐나?	미니 공사 vs 전면 리모델링	워밍업 설정 vs 실제 SFT 설정
레이어 구조	Q10	반복 적용되나?	아파트 각 층	레이어별 독립 A/B 파라미터
용량 조절	Q11	왜 파라미터가 급감하나?	축소 스케치	Rank(r), 저랭크 분해
	Q12	실제 결정 사항은?	리모델링 범위·강도	target_modules + r

한 문장 정리	Q13	한 문장 요약	포스트잇 + 스케치	PEFT
실전 적용	Q14	여러 Adapter 를 동시에 쓸 수 있나?	이미지 vs 언어모델 합성	Adapter 간 간섭(Interference)
	Q15	파라미터는 항상 재현되나?	같은 트레이싱지, 다른 손	재현성 조건(시드·데이터·하이퍼파라미터)
	Q16	결국 성능을 가르는 건 뭔가?	같은 도구, 다른 그림	데이터 구성·도메인 지식

Q1. LoRA 는 15 억 개 파라미터 중 일부를 골라서 학습하는 건가요?

아닙니다. 이 오해가 가장 흔합니다.

Base Model 을 두꺼운 원서 한 권이라고 생각해 보겠습니다. LoRA 는 이 책의 문장을 지우고 다시 쓰는 게 아니라, 책은 그대로 둔 채 필요한 페이지 옆에 포스트잇 메모를 붙이는 것에 가깝습니다. 책의 원문(기존 15 억 개 파라미터)은 손대지 않고 그대로 동결(Frozen)해 두고, 특정 Layer 옆에 작은 학습용 파라미터(LoRA Adapter)를 새로 추가해서 그것만 학습시킵니다.

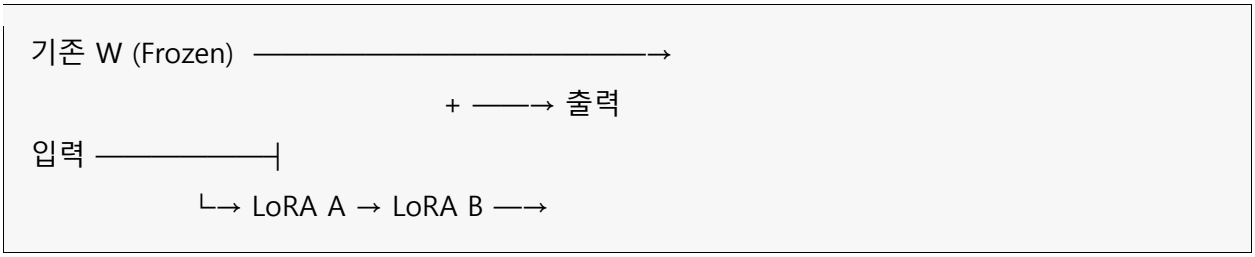
```
Base Qwen 1.5B
기존 Parameter
      ↓
Frozen (원문, 손대지 않음)
      +
LoRA Adapter (포스트잇 메모, 여기만 새로 씀)
      ↓
Trainable
```

즉 "15 억 개 중 일부를 고른다"가 아니라 "15 억 개는 그대로 두고, 새 파라미터를 추가로 얹는다"는 것이 정확한 이해입니다.

Q2. 그럼 기존 모델 옆에 어댑터를 병렬로 붙이는 개념인가요?

네, 이렇게 이해하면 직관적입니다. 비유하자면 기존 고속도로 옆에 작은 우회로를 하나 더 뚫는 것과 비슷합니다.

기존 Linear Layer(고속도로 W)는 폐쇄하지 않고 그대로 차량(입력)이 지나가게 둡니다. 다만 그 옆에 작고 좁은 우회로(A → B 라는 두 개의 작은 행렬)를 새로 만들어서, 이 길을 지나가는 학습된 변화량만 만들어냅니다. 그리고 두 길의 결과는 목적지(출력)에서 합쳐집니다.



기존 모델의 출력
+
LoRA 가 학습한 변화량
↓
최종 출력

다만 "병렬"이라고 해서 완전히 별개의 모델 두 개가 따로 도는 것은 아닙니다. 같은 Layer 안에서 기존 연산 옆에 작은 추가 경로가 붙고, 그 결과가 더해지는 정도로 이해하면 됩니다.

Q3. 그럼 기존 파라미터는 그대로 두고 새 파라미터만 얹어도 되는 근거는 뭔가요?

이건 여러 방법 중 하나를 임의로 고른 게 아니라, "파인튜닝으로 실제 필요한 변화는 원본보다 훨씬 단순하다"는 것이 실험으로 확인되었기 때문에 정당화되는 방식입니다. 이걸 저랭크 가설(Low Intrinsic Rank Hypothesis)이라고 부릅니다.

이미 아주 정교하게 완성된 대형 벽화(사전학습된 모델, W)가 있다고 해보겠습니다. 이 벽화 속 인물의 표정을 조금 더 부드럽게 바꾸고 싶습니다. 이때 벽화 전체를 다시 그릴 필요는 없습니다. 그림 위에 반투명한 트레이싱지 한 장을 덮고, 눈매와 입꼬리 부근에 몇 개의 얇은 보정선만 그어주면, 원래 그림과 겹쳐 보일 때 표정이 확 부드러워진 것처럼 보입니다.

여기서 W 는 Weight(가중치 행렬)의 약자입니다. 이 표기는, 신경망의 Linear(Dense) Layer 는 수식으로 y = Wx + b 형태로 계산되는데, 여기서 x 는 입력 벡터, y 는 출력 벡터, W 가 바로 입력을 출력으로 변환해주는 행렬입니다. 이 W 의 각 원소는 "입력의 어떤 요소가 출력에 얼마나 영향을

주는지"를 나타내는 값이라서 가중치(Weight)라고 부르고, 딥러닝 논문·코드에서 관례적으로 W 라는 기호로 표기합니다.
의미를 정리하면, 앞서 다뤘던 q_proj, v_proj 같은 Linear Layer 하나하나가 전부 이 W 를 하나씩 갖고 있고, 그 W 가 바로 "학습을 통해 조정되는 실제 숫자 값들의 행렬"입니다. 우리가 계속 써온 벽화·고속도로 비유에서 "벽화 자체" 또는 "기존 고속도로"에 해당하는 게 바로 이 W 이고, LoRA 는 이 W 는 건드리지 않고(Frozen) 그 옆에 작은 보정 행렬(A, B)만 추가하는 것이었습니다.

```
완성된 벽화 (W, Frozen)
+
트레이싱지 위 몇 개의 보정선 (저랭크 A → B)
↓
표정이 살짝 부드러워진 벽화 (최종 출력)
```

"표정을 부드럽게 바꾸는 데 필요한 정보"는 벽화 전체의 복잡도(수백만 개의 붓터치)에 비하면 훨씬 단순하기 때문에, 몇 개의 보정선만으로 충분합니다. 마찬가지로 파인튜닝에서 실제로 "바뀌어야 하는 부분(ΔW)"도 모델 전체의 복잡도에 비하면 몇 가지 굵직한 방향성만 담고 있는 경우가 대부분입니다.

> **NOTE** LoRA 논문은 보정선을 아주 적게 줄여도(Rank r 을 작게 잡아도) 벽화 전체를 다시 그린 것(Full Fine-Tuning)과 거의 비슷한 결과가 나온다는 것을 실험으로 확인했습니다. 사전학습된 모델일수록 이미 완성도가 높아서, 새 과제에 맞게 "살짝만 손봐도" 충분하다는 뜻입니다.

Q4. 그럼 LoRA 로 얹는 새 파라미터의 '값'은 어떻게 정해지나요? 베이스 모델마다 다른가요?

새 파라미터(A, B)의 값은 사람이 미리 정해서 넣는 게 아니라 학습 과정에서 자동으로 결정되고, 베이스 모델마다 당연히 다릅니다. 트레이싱지의 보정선은 그 벽화를 직접 보면서 그은 것이기 때문에, 다른 벽화에 그대로 옮겨 붙이면 안 맞는 것과 같은 이치입니다.

초기화 시점 : A 는 작은 무작위 값, B 는 전부 0 으로 시작합니다. B 가 0 이면 A×B 도 0 이 되어, 학습 시작 시점에는 보정 효과가 전혀 없는 상태(원본 모델과 완전히 동일한 상태)에서 출발합니다.

학습 시점 : forward → loss 계산 → backward → optimizer step 을 반복하면서, 준비한 데이터에 맞게 A, B 의 값이 조금씩 조정됩니다. 즉 새 파라미터의 내용은 학습 데이터와 역전파 알고리즘이 자동으로 채워 넣는 것입니다.

베이스 모델마다 값이 달라지는 이유는 두 가지입니다. 첫째, A·B 의 크기(shape) 자체가 보정 대상인 W 의 크기에 맞춰 정해지는데, 이 W 의 크기가 베이스 모델마다 다릅니다. 둘째, A·B 가 담아야 할 내용(보정 방향) 자체가 W 의 실제 값에 맞춰 결정되는데, 그 W 의 값이 베이스 모델마다 완전히 다르므로 같은 target_modules 를 골랐어도 학습된 A·B 값은 전혀 다른 내용이 됩니다.

> **NOTE** 같은 베이스 모델이라도 학습에 쓰는 데이터가 다르면(번역용 vs 요약용) 그 목적에 맞게 A, B 가 서로 다른 값으로 수렴합니다. 즉 '베이스 모델 + 학습 데이터'라는 두 조건이 합쳐져서 LoRA 어댑터의 실제 내용이 최종적으로 결정됩니다.

Q5. 그런데 학습시키기 전에 LoRA 를 붙이기만 해도 파라미터 개수가 바로 줄어들어 보이는데요?

실습에서 확인한 것은 "파라미터가 새로 생겨나는 시점"이었습니다. 하지만 "그 파라미터의 값이 의미 있게 채워지는 시점"은 서로 다릅니다. 즉, 새 파라미터(A, B)의 "값"은 사람이 미리 정해서 넣는 게 아니라 학습 과정에서 자동으로 결정되고, 베이스 모델마다 당연히 다릅니다. 하지만, "파라미터가 새로 생겨나는 시점"과 "그 파라미터의 값이 의미 있게 채워지는 시점"은 차이가 있다는 것입니다. LoRa 를 붙이는 순간 생기는 것은 전자이고, 학습 과정은 후자입니다.

LoRA 를 붙이는 것은 미술 시간에 그림 위에 투명한 트레이싱지를 테이프로 붙여놓는 것과 같습니다. 종이의 크기(A, B 의 shape)는 즉시 정해집니다. 어느 벽화의 어느 부분(target_modules) 위에, 얼마나 큰 종이(rank r)를 올릴지는 코드로 지정하는 순간 바로 확정되기 때문입니다. model.print_trainable_parameters()가 보여주는 숫자는 바로 이 종이 크기(총 슬롯 개수)를 세어서 보여주는 것입니다. 다만 이 시점의 트레이싱지에는 아직 아무 보정선도 그려져 있지 않습니다.

① 붙이는 순간 (구조 결정)
target_modules + r → A·B 의 크기·개수 즉시 확정
→ print_trainable_parameters()가 세는 것은 바로 이 '슬롯 개수'

② 학습이 진행되는 동안 (값 결정)
forward → loss → backward → optimizer step 반복
→ 빈 트레이싱지 위에 실제 보정선(A·B 의 구체적 수치)이 서서히 채워짐

정리하면, 종이를 붙이는 것(구조 생성)은 한 번의 동작으로 즉시 끝나는 일이고, 종이 위에 쓸모 있는 그림을 그리는 것(값 학습)은 정답과 비교하며 여러 번 고쳐 그리는 반복 작업입니다. 두 가지는 걸리는 시간도, 결정되는 방식도 완전히 다른 별개의 단계입니다.

Q6. 그럼 이 어댑터를 15 억 개 파라미터 전체에 다 붙이는 건가요?

아닙니다. 이번엔 건물 리모델링 비유가 잘 맞습니다.

건물 전체(Base Model)를 헐고 새로 짓는 게 아니라, "이 방과 저 방만 리모델링하겠다"고 미리 방 목록을 정하는 것과 같습니다. 이 방 목록을 코드에서는 target_modules 로 지정합니다.

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
```

이렇게 설정하면 실제로는 다음처럼 됩니다.

```
Qwen Transformer / Self Attention
 ├── q_proj   └── LoRA A/B   (리모델링 하는 방)
 ├── k_proj   └── LoRA 없음  (그대로 두는 방)
 ├── v_proj   └── LoRA A/B   (리모델링 하는 방)
 └── o_proj   └── LoRA 없음  (그대로 두는 방)
```

Base Parameter 는 전체를 동결하고, LoRA Adapter 는 우리가 지정한 방(Layer)에만 추가로 설치되는 것입니다.

Q7. 그럼 q_proj, k_proj, v_proj 같은 방(위치)은 누가 정하나요?

여기서 두 가지를 구분해야 합니다. 건물의 설계도는 이미 정해져 있고, 우리는 그중 어느 방을 고칠지만 선택할 수 있습니다.

q_proj, k_proj, v_proj 가 모델 어디에 존재하는지는 Qwen 을 설계할 때부터 이미 정해진 구조입니다. Self-Attention 에는 원래 다음과 같은 구조가 있습니다.



LoRA 를 적용하는 사람이 정하는 것은 딱 하나, "이 중 어느 방(Layer)에 어댑터를 설치할 것인가"입니다. 그게 target_modules 입니다.

Q8. Q, K, V 는 각각 무엇을 하는 건가요?

도서관에서 책을 찾는 상황을 떠올리면 쉽습니다.

Q (Query) : "내가 지금 찾고 싶은 키워드"입니다. 현재 토큰이 무엇을 알고 싶은가를 표현합니다.

K (Key) : "각 책 표지에 붙은 색인 태그"입니다. 다른 토큰이 나는 이런 정보를 갖고 있다는 걸 표현합니다.

V (Value) : "그 책을 실제로 펼쳤을 때 담긴 내용"입니다. 검색으로 선택된 토큰이 실제로 전달할 정보입니다.



```
해당 Token 들의 Value 활용 (실제 책 내용을 가져옴)
```

그래서 Q/K/V Projection 은 "어떤 정보를 어떻게 찾아 쓸지"를 결정하는 지점이라, Transformer 에서 가장 중요한 조정 포인트 중 하나입니다.

Q9. 그런데 왜 예제에서는 q_proj, v_proj 만 골랐나요?

가벼운 교육용 설정이기 때문입니다. 비유하자면 "방 두 개만 살짝 도배하는 미니 공사"로 LoRA 의 원리를 먼저 체감해보는 단계입니다.

```
target_modules=["q_proj", "v_proj"]
```

이렇게 범위를 좁히면 추가되는 Trainable Parameter 가 적어서, 무엇이 학습되고 무엇이 학습되지 않는지 비교해서 관찰하기 좋습니다. 다만 "LoRA 는 반드시 Q 와 V 에만 적용해야 한다"는 규칙은 아닙니다. 실제 SFT 학습 코드에서는 Attention 전체와 MLM 까지 넓혀서 "집 전체를 손보는 전면 리모델링" 수준으로 설정했습니다.

```
target_modules=[
    "q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj",
]
```

```
워밍업 실습                  →  실제 SFT
  Q + V                      →  Q+K+V+O + MLP Projection
                                        ↓
LoRA 원리를 가볍게 체감  →  더 많은 학습 용량 확보
```

Q10. q_proj 라고 지정하면, Qwen 안의 q_proj 하나에만 어댑터가 붙나요?

아닙니다. 같은 평면도의 아파트가 여러 층 쌓여 있는 건물을 떠올리면 됩니다.

Transformer 는 동일한 구조의 Block(층)이 여러 번 반복되는 구조입니다.

```
Qwen
Transformer Layer 0  └── Self Attention  └── q_proj
Transformer Layer 1  └── Self Attention  └── q_proj
Transformer Layer 2  └── Self Attention  └── q_proj
...
Transformer Layer N  └── Self Attention  └── q_proj
```

target_modules=["q_proj"]라고 지정하면, "이름이 q_proj 인 방은 모든 층에 각각 설치해 달라"는 뜻이 됩니다. 마치 "현관에 스마트락을 설치해 달라"고 하면, 1 층 현관 하나가 아니라 모든 층의 현관에 각각 별도로 스마트락이 설치되는 것과 같습니다.

```
Layer 0 q_proj  └── LoRA A/B
Layer 1 q_proj  └── LoRA A/B
Layer 2 q_proj  └── LoRA A/B
...
```

중요한 점은, 각 층의 스마트락(A/B 파라미터)이 서로 같은 열쇠를 공유하지 않는다는 것입니다. 구조와 Rank 설정은 동일해도, 각 Layer 마다 별도의 A/B 파라미터가 존재하고 독립적으로 학습됩니다.

Q11. LoRA 에서 학습 파라미터가 크게 줄어드는 원리는 뭔가요?

여기서 r(Rank)이 핵심입니다. 비유하자면, 물감의 기본 색을 몇 개까지 쓸 수 있게 허락할 것인가와 같습니다.

팔레트에 수백 가지 색깔이 놓여 있는 것처럼 보여도, 사실 빨강·파랑·노랑 같은 몇 가지 기본 물감만 있으면 그걸 섞어서 거의 모든 색을 만들어낼 수 있습니다. 이때 "실제로 필요한 기본 물감의 최소 개수"가 바로 Rank(계수)입니다. 팔레트에 놓인 색깔 종류(겉보기 크기)는 수백 가지여도, 그걸 만드는 데 필요한 진짜 기본 재료는 몇 개 안 되는 것입니다.

이걸 행렬로 옮겨서 설명하면, 어떤 행렬이 아무리 커 보여도(가로 1536, 세로 1536 처럼), 그 안의 값들이 서로 완전히 독립적이지 않고 몇 가지 기본 패턴의 조합으로만 이루어져 있을 수 있습니다. 이때 "몇 가지 기본 패턴"의 개수가 그 행렬의 Rank 입니다. Rank 가 낮다는 것은 "겉보기엔 복잡해

보여도 실제 정보는 단순하다"는 뜻이고, Rank 가 높다는 것은 "정말로 다양하고 독립적인 정보를 담고 있다"는 뜻입니다.

여기서 Rank 와 r 을 헷갈리지 않으려면 역할을 구분해야 합니다. Rank 는 이미 만들어진 팔레트(행렬)를 보고 "여기 실제로 몇 가지 기본 물감이 쓰였나" 관찰해서 알아내는 값입니다. 반면 r 은 LoRA 의 보정 팔레트(A, B)를 만들기도 전에 우리가 미리 "이번엔 기본 물감을 최대 몇 개까지만 쓰겠다"고 정해두는 설정값입니다. 즉 Rank 는 결과를 보고 측정하는 것이고, r 은 결과가 나오기 전에 미리 정해두는 한도라는 점이 다릅니다.

LoRA 의 r 이 이 개념과 어떻게 연결되는지 보면, LoRA 에서 조정해야 하는 "변화량(ΔW)"이 저랭크 가설에 따라 실제로는 매우 단순하다고 했었는데, r 은 바로 "이 변화량을 표현하는 데 기본 물감을 몇 개까지 쓰게 허락할 것인가"를 미리 정하는 값입니다. 그리고 이렇게 만들어진 보정 팔레트(ΔW)의 실제 Rank 는 이 r 을 넘지 않습니다.

이제 이 물감 개수 제한이 실제로 파라미터 개수를 얼마나 줄여주는지 숫자로 확인해 보겠습니다. 원래 Linear Layer 가 1536 × 1536 크기라고 가정해 보겠습니다.

```
W: 1536 × 1536 = 2,359,296 Parameters
```

전체를 직접 학습하려면 이만큼의 숫자를 다뤄야 합니다. LoRA 는 이 변화량을, 물감을 섞어내는 두 개의 홀쭉한 보정 팔레트(A, B)로 대신 표현합니다. r=8 이라면,

```
A: 8 × 1536
B: 1536 × 8

(8 × 1536) + (1536 × 8) = 24,576 Parameters
```

기존 236 만 개를 직접 손대는 대신, 약 2.5 만 개의 물감(파라미터)만으로 그 Layer 에 필요한 변화를 대신 표현해내는 것입니다. r 이 커질수록 사용할 수 있는 기본 물감의 가짓수가 늘어나고, 그만큼 표현할 수 있는 색조 변화의 폭(학습 용량)도 커집니다.

```
r=4  → 기본 물감 4 개만 사용 → 표현 가능한 변화 단순함 → Trainable Parameter 적음
r=8  → 조금 더 다양한 색조 표현 가능
r=16 → 더 다양한 색조 표현 가능
```

r=32 → 기본 물감 32 개 사용 → 표현 가능한 변화 정교함 → Trainable Parameter 많음

Q12. 그럼 LoRA 를 쓸 때 실제로 결정해야 하는 건 뭔가요?

크게 두 가지, 즉 리모델링할 방의 개수와 공사의 강도입니다.

① 어디에 어댑터를 붙일 것인가 (target_modules) : q_proj 만? v_proj 까지? Attention 전체? MLP 까지?

② 어댑터를 얼마나 크게 만들 것인가 (r, Rank) : 4? 8? 16? 32?

적은 Target + 낮은 Rank
↓
Trainable Parameter ↓, 메모리·비용 ↓, 학습 용량 ↓
(방 몇 개만, 도배 정도의 가벼운 공사)

많은 Target + 높은 Rank
↓
Trainable Parameter ↑, 메모리·비용 ↑, 학습 용량 ↑
(전체 리모델링, 구조까지 크게 손보는 공사)

즉 LoRA 설정은 결국 "얼마나 넓은 범위를, 얼마나 깊게 바꿀 것인가"를 정하는 선택이라고 보시면 됩니다.

Q13. LoRA 를 한 문장으로 정리하면?

> **NOTE** LoRA 는 Base Model 의 기존 파라미터는 손대지 않고 모두 동결한 상태에서, Transformer 의 특정 Layer 옆에 작은 저랭크 어댑터(포스트잇)를 추가하고 그 어댑터만 학습시켜, 적은 비용으로 모델의 행동을 조정하는 PEFT 기법입니다.

Qwen Base Model
1.5B Parameter

```
(Frozen)
기존 연산 경로(Frozen)  +  LoRA Adapter(A → B)
↓
최종 출력
```

Q14. LoRA 모델을 동시에 여러 개 Adapter 를 붙여서 사용하기도 하나요?

이미지 생성 모델(Stable Diffusion 등)에서는 인물 스타일 LoRA 와 배경 스타일 LoRA 처럼, 서로 다른 어댑터의 가중치를 조절해 동시에 합성(merge)하는 방식이 실제로 널리 쓰입니다. 픽셀 단위로 겹쳐도 "그럴듯한 이미지"가 나오기 쉽기 때문입니다.

언어모델은 사정이 다릅니다. 서로 다른 문체·규칙·도메인 지식을 학습한 두 어댑터를 동시에 얹으면, 어떤 어댑터의 지시를 따를지 문장 단위로 충돌(간섭)이 발생해, 답변이 뒤섞이거나 앞뒤가 맞지 않는 경우가 흔합니다.

> **NOTE** 그래서 언어모델에서는 이미지 모델처럼 여러 LoRA Adapter 를 동시에 얹어 쓰는 방식이 실전에서는 잘 사용되지 않습니다. 여러 목적이 필요하다면 처음부터 하나의 데이터셋으로 함께 학습시키는 방식을 더 선호합니다.

15. 모델이 동일하면, LoRA 를 붙였을 때 파라미터도 항상 동일한가요?

같은 벽화(Base 모델) 위에 같은 자리(target_modules)에 같은 크기(r)의 트레이싱지를 붙이는 것까지는 항상 동일합니다. 하지만 그 위에 실제로 그려지는 보정선, 즉 A·B 의 최종 값은 다를 수 있습니다.

동일한 파라미터 값이 나오려면 Base 모델 가중치, target_modules, r·alpha, A 의 무작위 초기화 시드, 학습 데이터와 그 순서, 학습률·배치 크기·epoch 같은 하이퍼파라미터, 연산 정밀도까지 필요한 조건이 전부 일치해야 합니다.

> **NOTE** 이 조건들이 모두 충족되면 이론적으로는 동일한 파라미터가 나올 수 있습니다. 하지만 하나라도 어긋나면 결과는 달라지므로, 완전한 재현은 이론적으로만 가능한 이야기에 가깝습니다.

Q16. LoRA 성능에 영향을 미치는 것이 사실 데이터 구성 아닌가요? 그럼 도메인 지식이 더 중요해지는 것 아닌가요?

훌륭한 질문이며, 정확한 관찰입니다. r, target_modules, learning rate 같은 하이퍼파라미터는 이미 널리 검증된 범위 안에서 고르면 성능 차이가 크지 않습니다.

반면 어떤 질문-답변 쌍을 얼마나 균형 있게, 어떤 형식으로, 어떤 도메인 지식을 담아 구성하느냐는 결과 모델의 실제 품질을 좌우합니다.

> **NOTE** 그래서 LoRA 를 다루는 실무에서는 알고리즘 자체를 더 깊이 파는 것보다, 현장 도메인을 잘 아는 사람이 데이터를 어떻게 설계하느냐가 성능을 훨씬 크게 좌우합니다.

맺음말

앞서 진행했던 "PEFT 적용 전/후 Parameter 비교" 실습의 의미도 여기까지 이해하면 명확해집니다. 그때 확인한 것은 "15 억 개 중 약 1,000 여개를 골랐다"가 아니라, "15 억 개의 Base Parameter 는 그대로 동결하고, 우리가 선택한 Layer 들에 작은 새 Trainable Parameter 를 추가했다"는 것을 실제 우리가 실습 코드에서 확인할 수 있었습니다.

Q1~Q2 에서 LoRA 가 무엇을 새로 추가하는지, Q3 에서 왜 원본을 손대지 않고 얹기만 해도 되는지(저랭크 가설), Q4~Q5 에서 그 값이 언제·어떻게 정해지는지(구조 생성과 값 학습의 차이), Q6~Q7 에서 어디에 붙이는지, Q8~Q9 에서 Q/K/V 와 적용 범위를, Q10 에서 층마다 왜 독립적으로 학습되는지, Q11~Q12 에서 Rank 로 학습 용량을 어떻게 조절하는지 차례로 확인했습니다.

이어서 Q14 에서는 여러 LoRA Adapter 를 동시에 얹는 방식이 이미지 모델과 달리 언어모델에서는 왜 잘 쓰이지 않는지, Q15 에서는 같은 모델·같은 설정이라도 파라미터 값이 항상 같지는 않은 이유는 무엇인지, Q16 에서는 결국 LoRA 성능을 가르는 것이 알고리즘 자체보다 데이터 구성과 현장 도메인 지식이라는 점까지 살펴보았습니다.

프론티어 모델 자체를 새로 학습시키는 영역(사전학습, 초거대 아키텍처 설계)에서는 알고리즘·컴퓨팅 자원의 격차가 여전히 큰 승부처입니다. 다만 지금 이 가이드처럼 "이미 나온 Base 모델을 현장에 맞게 특화시키는" 현장 실전 영역에서는, LoRA 뿐만이 아니라 LLM 관련된 기술들은 오픈된 것들이고, 현장에서 적용하며 쌓이는 도메인 지식이 인공지능 승패를 판가름할 것입니다.

결국 LoRA 뿐 아니라 지금 우리가 다루는 LLM 기술 대부분 " Transformer 구조, PEFT 기법, 학습 도구 "은 이미 논문과 오픈소스로 공개되어 누구나 같은 도구를 손에 쥘 수 있습니다. 같은 도구를 쓰는 이상, 승부는 트레이싱지를 붙이는 방법이 아니라 그 위에 무엇을 그리느냐 " 즉 현장에서 쌓은 도메인 지식과 데이터 구성 " 에서 갈립니다.
이 가이드가 다룬 LoRA 의 구조는 출발점일 뿐이며, 진짜 경쟁력은 여러분의 현장이 가진 데이터를 이 구조 위에 어떻게 얹느냐에 달려 있습니다.
앞으로 펼쳐질 AI Service 에서 AI Leader 로 성장해 나갈, 여러분들의 여정을 응원합니다.

SKALA 임성열 교수 드림
Ph.D.